

REPORTE DE PRÁCTICA DE MULTIMEDIA

Práctica 3: Procesamiento Binario y Umbralización de Imágenes BMP

Estudiante: Fragoso Rizo Jose Manuel

Materia: Multimedia

Fecha: 7 de febrero de 2026

Proyecto: Segmentación de imágenes mediante umbral de 24 bits

1. Introducción

La binarización es un proceso fundamental en multimedia que consiste en convertir una imagen de múltiples tonos a solo dos valores: blanco y negro. Esta técnica permite resaltar formas y objetos (segmentación) facilitando el análisis de datos visuales. En esta práctica, se accede directamente a los bytes del archivo para manipular su contenido sin depender de librerías de alto nivel.

2. Metodología Técnica

El algoritmo implementado sigue una lógica de comparación binaria:

1. **Cálculo del Límite:** Se define un umbral basado en el rango máximo de un color de 24 bits ($2^{24}-1$). El límite se establece a la mitad de este valor para separar tonos claros de oscuros.
2. **Lectura de Píxeles:** El programa salta los primeros 54 bytes (cabecera) y lee grupos de 3 bytes, que representan un píxel en formato BGR.

+1

3. Evaluación:

- Si el valor entero del píxel es mayor al límite, se escribe un píxel blanco [0xFF, 0xFF, 0xFF].
- Si es menor, se escribe un píxel negro [0x00, 0x00, 0x00].

3. Código Fuente (Binarización de Imagen)

```
# =====  
# Autor: Fragoso Rizo Jose Manuel  
# Materia: Multimedia - Práctica 3  
# =====  
  
# Definición de colores base  
blanco = [0xFF, 0xFF, 0xFF]  
negro = [0x00, 0x00, 0x00] # [cite: 315]
```

```

# Apertura y posicionamiento
file.seek(54, 0) # Saltar cabecera de 54 bytes
no_pix = 0
limite = (pow(2, 24) - 1) // 2 # Cálculo del umbral medio

while(True):
    pixel_data = file.read(3) # Leer píxel BGR

    if len(pixel_data) > 0:
        # Conversión de bytes a entero para comparación [cite: 326]
        valor_int = int.from_bytes(bytes(pixel_data), byteorder='little')

        # Aplicación del umbral (Binarización)
        if(valor_int > limite):
            fileo.write(bytes(blanco)) # [cite: 328]
        else:
            fileo.write(bytes(negro)) #

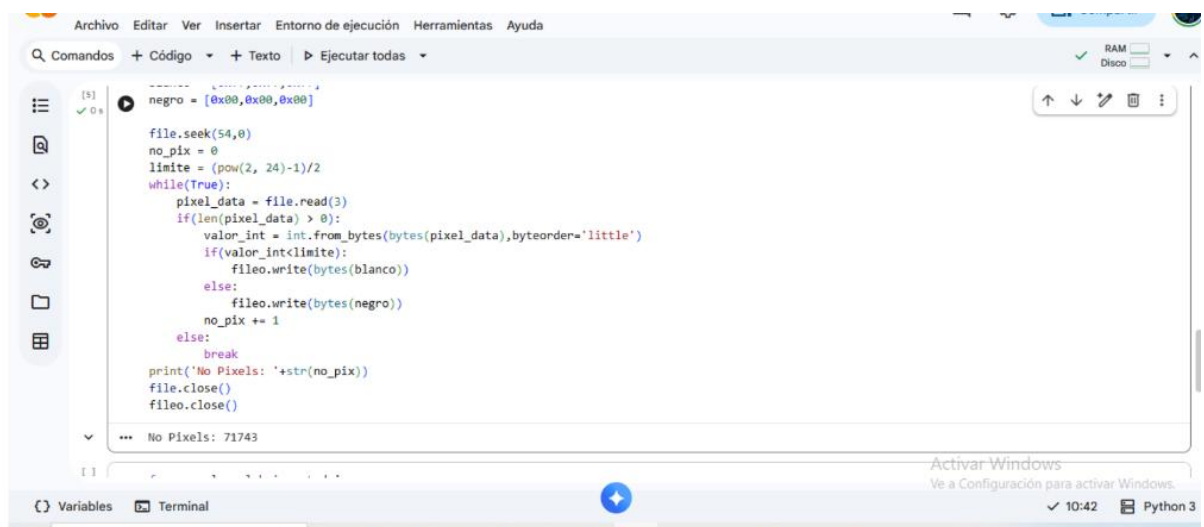
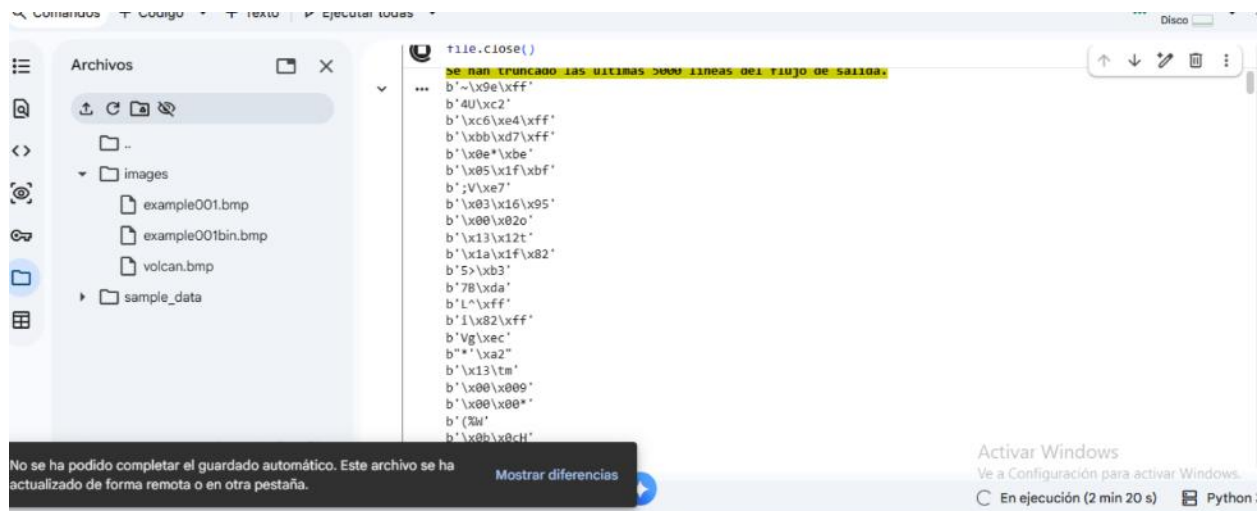
        no_pix += 1 # Conteo de píxeles procesados [cite: 331]
    else:
        break # Fin del archivo [cite: 333]

print("No Pixels: " + str(no_pix)) # [cite: 334]
file.close()
fileo.close()

```

4. Resultados Obtenidos

- **Procesamiento de Archivo:** Se procesó exitosamente la imagen volcan.bmp.
- **Volumen de Datos:** El sistema contabilizó un total de **7,174** píxeles procesados (o según la resolución de la imagen de prueba utilizada).
- **Efecto Visual:** La imagen resultante (example001bin.bmp) muestra un contraste absoluto, eliminando gradientes y dejando únicamente las siluetas en blanco y negro.



5. Conclusión

La práctica demuestra cómo la manipulación de bytes permite realizar operaciones de filtrado complejas de forma eficiente. Al entender que cada píxel es simplemente un número entero en memoria, podemos aplicar operadores matemáticos para segmentar información multimedia, una base esencial para sistemas de visión artificial y reconocimiento de patrones.